

ReachLab: a reproducible synthetic collision-learning experiment in a planar robot workbench

AI-assisted technical report draft for review by Amogh Sarasam

October 8, 2026. Engineering technical report; not peer reviewed or submitted to a journal.

Abstract

ReachLab implements a browser workbench for exploring two-link planar robot kinematics, obstacle avoidance, configuration-space planning, and supervised collision classification without a physical robot. This report documents its mathematical model and a reproducible synthetic benchmark of a five-nearest-neighbor classifier. Joint configurations were generated using specified pseudorandom seeds; labels came from segment-to-circle collision geometry with link thickness. Three nested training sets containing 100, 500, and 1,500 configurations were evaluated against the same independently generated 3,000-configuration held-out set. Accuracy increased from 76.80% to 86.23% and 93.47%, while collision recall increased from 45.48% to 71.15% and 87.04%. The largest model nevertheless classified 106 of 818 colliding test configurations as free. These observations support treating the classifier as a fallible approximation for inspection and learning. Path acceptance instead uses geometry and a conservative displacement bound. Results apply to one simulated obstacle scene and sample distribution; they establish neither hardware safety nor educational effectiveness.

Motivation and related work

Learning robot geometry does not inherently require robot ownership. A small browser workbench can let a learner inspect how joint angles change an arm, how workspace obstacles become configuration-space obstacles, and how data-driven predictions differ from known geometry. ReachLab targets this narrow learning activity with a local computational model. Reducing ongoing network requirements is a design motivation, not a measured social outcome.

NCES reports that in 2021, 93% of U.S. children ages 3-18 had home internet access through a computer, 4% relied on smartphones, and 3% lacked home internet [1]. Its computer definition includes tablets. These historical national statistics describe differences in access; they do not measure robotics access or ReachLab's effect. A local workbench still needs a compatible device and initial distribution.

Browser robotics education is established prior work. VEXcode VR provides virtual robot coding without installation [2], and Cyberbotics described online Robotbenchmark challenges in 2017 [3]. Robot kinematics, configuration spaces, and grid-based planning are standard topics documented by Lynch and Park [4-6]. ReachLab therefore makes no claim to invent virtual robotics education, inverse kinematics, A*, or nearest-neighbor classification. Its contribution here is a specific inspectable implementation and reproducible experiment connecting those components.

Simulated geometry and kinematics

The benchmark arm has two revolute joints, link lengths $L_1=130$ and $L_2=100$, base $(0,0)$, and link radius 5. All distances are simulation units. Angles are radians. If q_1 is the first joint angle and q_2 is the second joint's relative angle, the elbow is

$$e=(L_1 \cos(q_1), L_1 \sin(q_1)),$$

and the tip is

$$p=(L_1 \cos(q_1)+L_2 \cos(q_1+q_2), L_1 \sin(q_1)+L_2 \sin(q_1+q_2)).$$

The analytic inverse implementation derives $\cos(q_2)=(x^2+y^2-L_1^2-L_2^2)/(2L_1L_2)$. A reachable target generally gives two elbow branches; targets outside the reachable annulus return no solution. Singular branches are merged within the implemented tolerance. This is ordinary planar kinematics, not an empirical robot calibration.

Each link is modeled as a segment thickened by its radius. A circular obstacle collides when its center-to-segment distance is no greater than the sum of obstacle and link radii, including a 10⁻⁹ comparison tolerance. The closest segment point uses a clamped orthogonal projection, covering both interior and endpoint contacts. Both links are tested. The benchmark scene contains circles (center x , center y , radius) at $(145,35,22)$, $(65,-110,24)$, and $(-105,75,18)$.

This label calculation is the reference rule for the chosen geometric model. It is not a reference measurement from a manufactured robot. The implementation omits self-collision, a separately sized base housing, out-of-plane geometry, joint torque, velocity, compliance, and sensing error.

Planning and conservative edge acceptance

The planner accepts joint coordinates within $[-\pi,\pi]$ and first rejects colliding endpoints. It attempts a direct joint-space edge, then searches an eight-connected bounded grid using A*. Its default resolution is 48 intervals per angle, producing 49×49 grid positions. Edge cost is Euclidean joint displacement; the heuristic is straight-line distance to the snapped goal. The implementation checks connectors between exact endpoints and grid nodes. It does not connect opposite boundaries of the angular grid: this bounded planner differs from its classifier's periodic metric. Resolution and conservative rejection can prevent a route from being found even when another continuous route exists.

Edge acceptance is stronger than isolated point sampling. For a candidate edge with joint changes d_1 and d_2 , the implementation selects $m=\max(1,\text{ceil}(\max(|d_1|,|d_2|)/0.015))$ subintervals. It bounds link-point movement over one subinterval by

$$B=(L_1|d_1|+L_2(|d_1|+|d_2|))/m.$$

This follows from bounding rotational movement by radius times angular displacement and adding movements of the two links. Every point within a subinterval lies within $B/2$ of a corresponding point at its nearer endpoint. Segment-to-obstacle distance cannot decrease by more than that displacement. Accordingly, every checked endpoint must clear each inflated obstacle by more than $B/2$ plus the numerical tolerance. Passing these checks conservatively certifies avoidance of the modeled circles

along the specified linear joint interpolation under the mathematical model. Uncertain edges are rejected rather than accepted.

This argument concerns ideal arithmetic and the specified geometry; implementation uses floating-point trigonometry. It does not certify hardware safety. The benchmark below evaluates collision classification, not planner completeness, path optimality across continuous motions, or timing. The learned model never supplies permission to accept an edge.

Learning experiment and reproducibility

The experiment asks how the measured performance of a fixed five-neighbor classifier changes across three specified training sizes in this one scene. It is a descriptive engineering comparison. No preregistered hypothesis, human study, or significance test is claimed.

`sampleDataset` draws q_1 and q_2 separately over $[-\pi, \pi)$ using a seeded linear congruential pseudorandom generator: state is updated by $1664525 \times \text{state} + 1013904223$ modulo 2^{32} . Training seed 314159 creates nested sets of 100, 500, and 1,500 configurations. Held-out seed 90210 creates 3,000 other configurations, reused across all models. Distinct seeds separate generation streams; this is not a proof of mathematical independence. Labels are computed before predictions using the same fixed scene's geometric rule. No held-out labels fit the classifier.

For query angles q and training angles s , neighbor ranking uses

$$D(q,s) = 2 - 2\cos(q_1 - s_1) + 2 - 2\cos(q_2 - s_2).$$

This is squared chord distance under the feature encoding $(\cos(q_1), \sin(q_1), \cos(q_2), \sin(q_2))$, avoiding a distance discontinuity at angular wraparound. Five neighbors vote with equal weight; at least half voting collision yields a collision label. Equal distances are resolved by original sample index. The reported collision probability is a neighbor vote fraction, not a calibrated probability. Nearest-neighbor classification and held-out evaluation follow established practice [7,8]; the implementation is JavaScript, not a scikit-learn execution.

From the ReachLab directory, with Node.js 22 or newer, run `npm test` to execute the included verification suite and `npm run benchmark` to regenerate `research/results/benchmark.json` and `benchmark.csv`. `scripts/benchmark.mjs` specifies the scene, seeds, sample counts, and k . `src/robotics.js` contains geometry, sampling, classification, and evaluation. The benchmark itself requires no downloaded dataset or external package. An unchanged implementation produces deterministic counts; modifying geometry or sampling creates a different experiment.

Results

Collision is the positive class. The held-out set contains 818 colliding and 2,182 free configurations. Accuracy is $(TP+TN)/N$; precision is $TP/(TP+FP)$; recall is $TP/(TP+FN)$. False-safe rate here means $FN/(TP+FN)$, the fraction of actual collisions mislabeled free, not the fraction of all predictions that are incorrect.

Training n	TP	TN	FP	FN	Accuracy	Collision precision	Collision recall	False-safe rate
100	372	1,932	250	446	76.80%	59.81%	45.48%	54.52%
500	582	2,005	177	236	86.23%	76.68%	71.15%	28.85%
1,500	712	2,092	90	106	93.47%	88.78%	87.04%	12.96%

All values derive from the saved benchmark counts. A constant-free classifier would score 72.73% accuracy on this set while detecting no collisions. Thus overall accuracy alone obscures important behavior. The observed accuracy increase from 100 to 1,500 examples is 16.67 percentage points, but the largest model still misses 106 collisions. Its 90 false positives also incorrectly mark free configurations as blocked. Both error types matter for interpreting a learned collision map.

Interpretation and limitations

Larger training sets performed better on the reported fixed test set. This does not establish that every additional sample improves performance, that the classifier generalizes to other scenes, or that it accelerates planning. There is one training seed, one held-out seed, one obstacle layout, and one fixed k . Nested training sets and a shared test set permit a direct comparison here, but they do not provide independent replications. Runtime and memory were not benchmarked.

Obstacle movement changes the target label function. Reusing a model after changing the scene would require new labels and evaluation. Uniform joint sampling may poorly represent a learner's actual interaction patterns or configurations near narrow collision boundaries. Future evaluation could predefine multiple scenes and seeds, assess boundary regions separately, and compare classifiers with simple baselines on a separate validation set before final testing.

The workbench's educational motivation remains untested. No students were recruited, no learning gains measured, and no deployment reach documented. No physical robot was controlled. The report's defensible outcome is a reproducible synthetic result illustrating why an approximate learned classifier and a geometry-based acceptance rule should have distinct roles.

References

1. NCES. Children's Internet Access at Home. Updated August 2023; 2021 ACS data. <https://nces.ed.gov/programs/coe/indicator/cch/access-to-computers-and-the-internet>
2. VEX Robotics. VEXcode VR setup and features. <https://www.vexrobotics.com/vexcode/install/vr>
3. Cyberbotics. Announcing Robotbenchmark. August 21, 2017. <https://cyberbotics.com/doc/blog/robotbenchmark>
4. Lynch, K. M., and Park, F. C. Modern Robotics: Mechanics, Planning, and Control. Official textbook resources. https://hades.mech.northwestern.edu/index.php/Modern_Robotics
5. Lynch and Park. 10.2.1. C-Space Obstacles. <https://modernrobotics.northwestern.edu/nu-gm-book-resource/10-2-c-space-obstacles/>
6. Lynch and Park. 10.4. Grid Methods for Motion Planning. <https://modernrobotics.northwestern.edu/nu-gm-book-resource/10-4-grid-methods-for-motion-planning/>

7. Scikit-learn developers. Nearest Neighbors. <https://scikit-learn.org/stable/modules/neighbors.html>

8. Scikit-learn developers. Cross-validation: evaluating estimator performance. https://scikit-learn.org/stable/modules/cross_validation.html

Contribution and status disclosure

AI assistance produced software, synthetic analysis, source notes, and this report draft. No claim is made that Amogh personally performed the documented implementation, experiments, or prose composition. Human review must verify the code, regenerate results, inspect references, and describe actual contributions before any authorship or submission representation. This draft is complete as a technical account of the saved synthetic experiment; it is not evidence of journal acceptance, competition entry, or independent student research.